UNIVERSITA' DEGLI STUDI DI MILANO BICOCCA

Intro to statistical learning

Lecture 9

Course of:

Statistics and data analysis

Federico De Guio - Martino Borsato

Intro

Context

- Nowadays **Artificial intelligence** is used in every field you can think of
  - From agriculture to real estate
  - From healthcare to entertainment
  - In every aspect of research

- **Machine learning and deep learning are everywhere**
  - Huge development in recent years also thanks to the progress in data storage technologies, hardware acceleration, …

- Cannot talk about data analysis without introducing some **elements of machine learning**

Outline of the second part of the course

- How to learn from data
- Regression vs classification, supervised vs unsupervised
- Data preparation and regularization methods
- Neural network as a universal approximator
- How to train a DNN
- CNN (and GNN)
- MCMC methods to fit a function to a dataset
- Associate an uncertainty to NN outputs
- Unfolding and interplay with ML

Books and references

- Not following a single book from beginning to end
  - Will try to give some precise reference to books and chapters lecture by lecture

- In general the material discussed in the upcoming lectures is included in the following books
  - **James** - "An Introduction to Statistical Learning"
  - **Prince** - "Understanding Deep Learning"Apri questo documento con ReadSpeaker docReader
  - **Goodfellow** - "Deep learning"
  - **Mackay** - "Information Theory, Inference and Learning Algorithms"Apri questo documento con ReadSpeaker docReader
  - Fernandez-Granda - "Probability and statistics for data science"

- Where available you find direct links to the books from the eLearning page of the course

Statistical learning

- James, cap.2
- Prince, cap.1

Statistical learning problems

- Very wide landscape
  - Customize an email spam detection system

- **Probabilistic classifier** built on top of multiple signal layers
  - Parsing
  - Feature extraction
  - Multiple specialized models
  - Aggregation
  - Total score and final decision

Statistical learning problems

- Very wide landscape
  - Customize an email spam detection system
  - Identify the numbers in a handwritten zip code

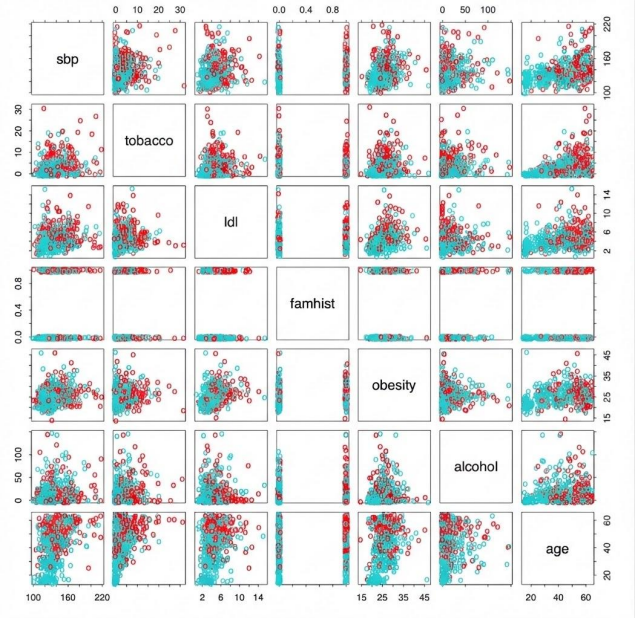
- **Image transformation and feature extraction**
  - Analysis of huge input dataset
  - Isolate and highlight edges, shapes
  - Classification of digits



Statistical learning problems

- Very wide landscape
  - Customize an email spam detection system
  - Identify the numbers in a handwritten zip code
  - Predict whether someone will have a heart attack on the basis of demographic, diet and clinical measurements

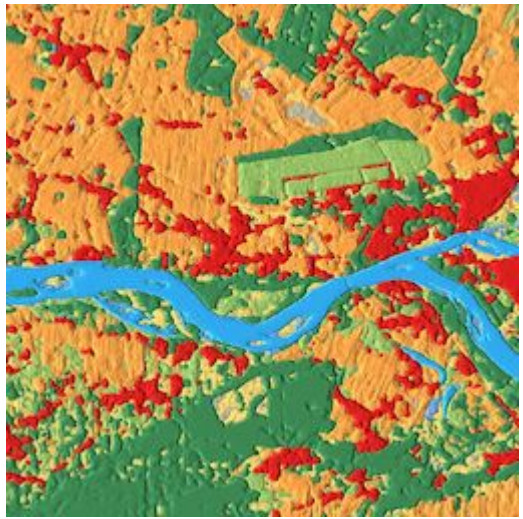
- **Analysis of correlations** between multiple features
  - Features go from habits to blood measurements



Statistical learning problems

- Very wide landscape
  - Customize an email spam detection system
  - Identify the numbers in a handwritten zip code
  - Predict whether someone will have a heart attack on the basis of demographic, diet and clinical measurements
  - Classify the pixels in a LANDSAT image

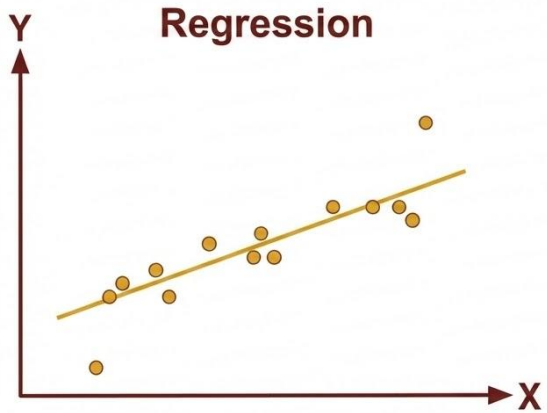
- **Image segmentation**
  - Assign each pixel to a given class
  - Environmental measurements
    - Deforestation
    - Glacier shrinking
    - Monitoring of wildfires
    - …



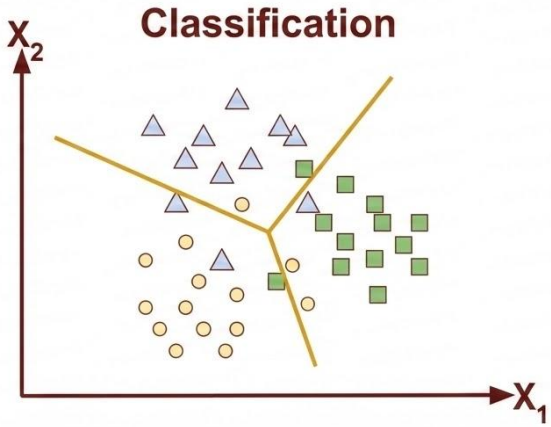
In all cases the objectives are

- Accurately **predict unseen cases**
- **Understand** which inputs affect the outcome, and how
- Assess the **quality of our predictions** and inferences

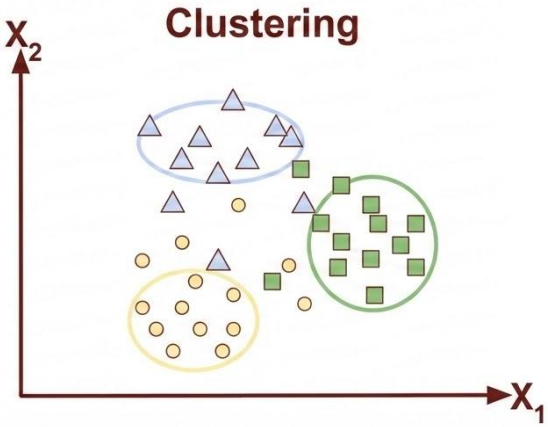
The data landscape: **three archetypes**



Estimate the **value** of a dependent variable from one or more independent variables

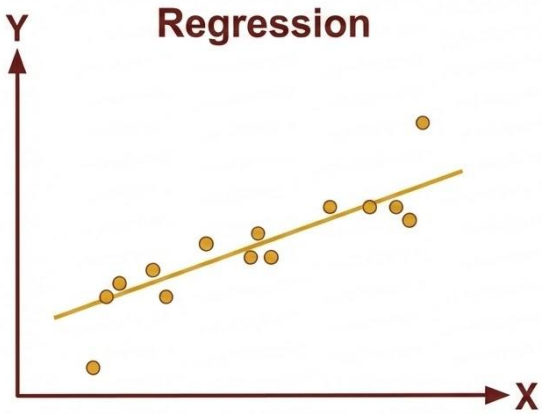


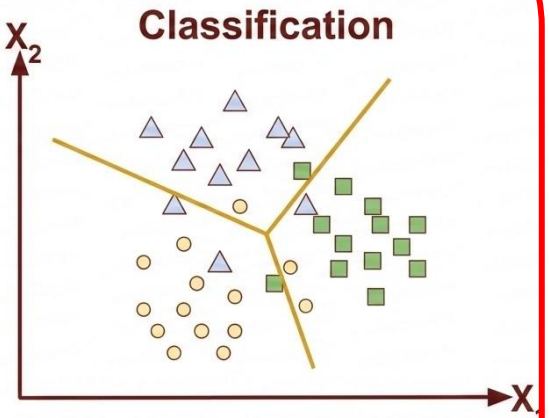
Identify the **class/group** the example belongs to based on its characteristics



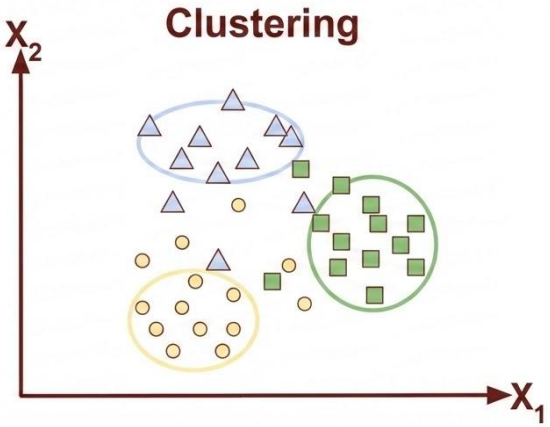
Group objects into classes of similar objects based on the similarity between each other

The data landscape: **three archetypes**





SUPERVISED
Build a function that maps input to output based on the relationship in a given labeled input-output pair examples, the training dataset



UNSUPERVISED
Find previously unknown patterns in dataset without pre-existing labels

The **supervised** learning problem

- **Outcome measurement Y** → we will call it **target** or **label**
- **Vector of p predictor measurements X** → we will call them **inputs** or **features**

- In the **regression** problem, Y is quantitative
- In the **classification** problem, Y takes values in a finite, unordered set

- We have data $(x_1, y_1), \ldots, (x_N, y_N)$. **These are observations of these measurements** → we will call them **examples** or **instances** or **events** and they are the **training data**

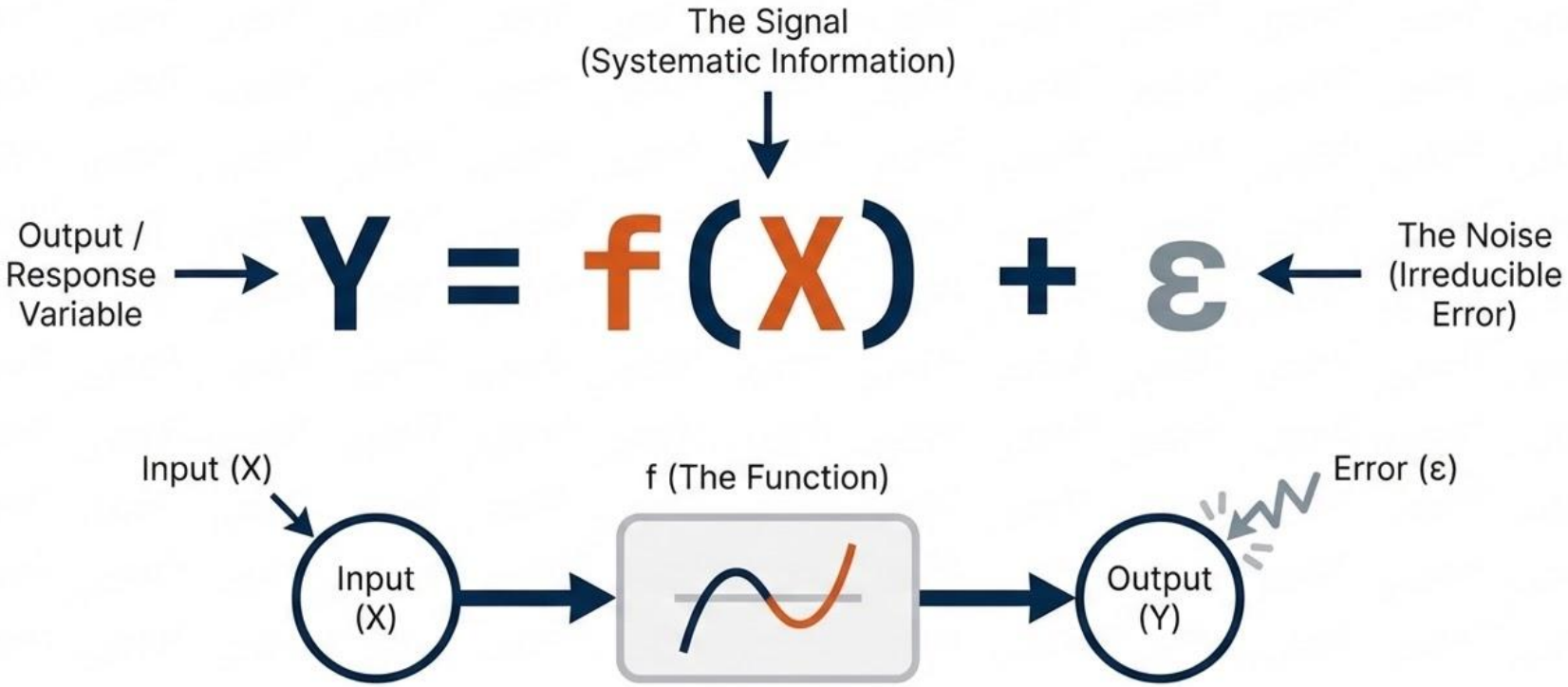
- On the basis of the training data we would like to:
  - **Accurately predict unseen test cases**
  - **Understand which inputs affect the outcome, and how**
  - **Assess the quality of our predictions and inferences**

The **unsupervised** learning problem

- **No known outcome variable**, just a set of features
- objective is more fuzzy
  - **find groups of samples that behave similarly**
  - **find features that behave similarly**
  - **find linear combinations of features with the most variation**

- Harder to know how well you are doing
- Unsupervised learning methods such as clustering or PCA can be used to **prepare the data** for a supervised learning step (see tomorrow's lecture)

In all cases we can trace back the goal to: **finding f(X)**



What is f(X) good for?

- With a good f we can **make predictions** of Y at new points X = x
- We can **understand which components** of $X = (X_1, X_2, \ldots, X_p)$ **are important** in explaining Y, and which are irrelevant
- Depending on the complexity of f, we may be able to understand how each component X_j of X affects Y

Example: classification of particles coming out from a collision at the LHC

- **We want to estimate the relationship between the event features X and the physical nature of the event Y**

- **X**: the high-level observables
  - invariant masses of particles, E_T^{miss}, angular correlations between particles, etc
- **f(X)**: contains the underlying physics (Standard Model)
  - Represent the information X provides about Y
  - Simplest case: want classify each event
    - Y=1 → higgs
    - Y=0 → background
- ϵ: irreducible noise
  - Detector resolution, pileup, etc

- **Goal:** use known data (simulations) to build $\hat{f}$ such that $\hat{Y} = \hat{f}(X)$ is the closest possible to **Y**
  - This is what we always do when looking for the law describing the data at best
  - Use $\hat{f}$ to decide what event to keep and what to discard on real data

What's statistical learning in the end? What's the difference with machine learning?

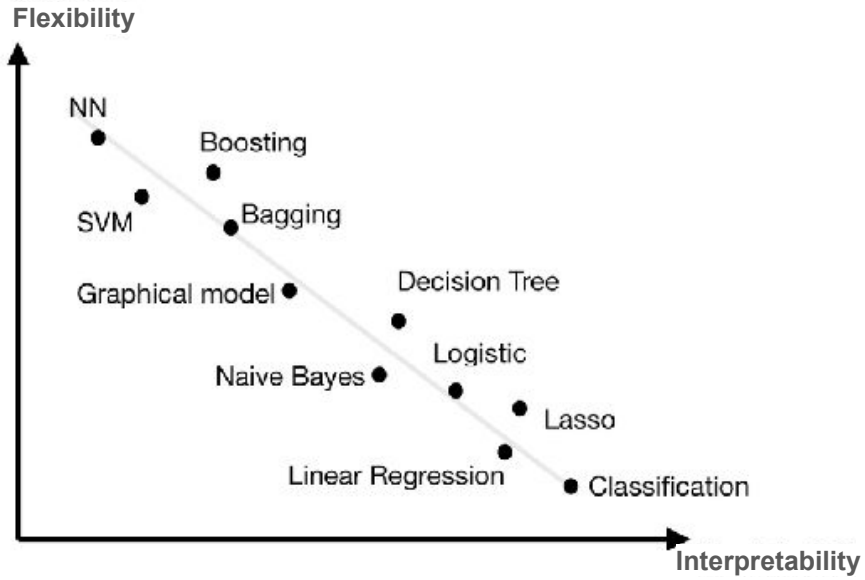
- Statistical learning refers to a **set of approaches for estimating f**
  - Statistical learning arose as a subfield of Statistics
- Machine learning arose as a subfield of Artificial Intelligence
- Much overlap between the two!
  - both fields focus on **supervised and unsupervised problems**
  - **Machine learning** has a greater emphasis on **large scale applications and prediction accuracy**
  - **Statistical learning** emphasizes models and their **interpretability, and precision and uncertainty**
- Distinction between the two is blurred

Model interpretability vs model flexibility

- **High flexibility:** the model follows well the data used to shape it → **risk overfitting**
- **High interpretability:** the model is simple, it can miss important structures present in the data → **risk underfitting**

- As methods get more flexible, they become harder to interpret

- The choice depends on the nature of the problem we want to solve:
  - **Inference**: understand the relationship between variables → **choose high interpretability**
  - **Prediction:** predict the outcome with high accuracy → **choose high flexibility**



Model accuracy

- In order to evaluate the **performance of a statistical learning method** on a given data set, we need some way to measure how well its predictions actually match the observed data
  - In the regression setting **MSE** is a common choice

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{f}(x_i))^2$$

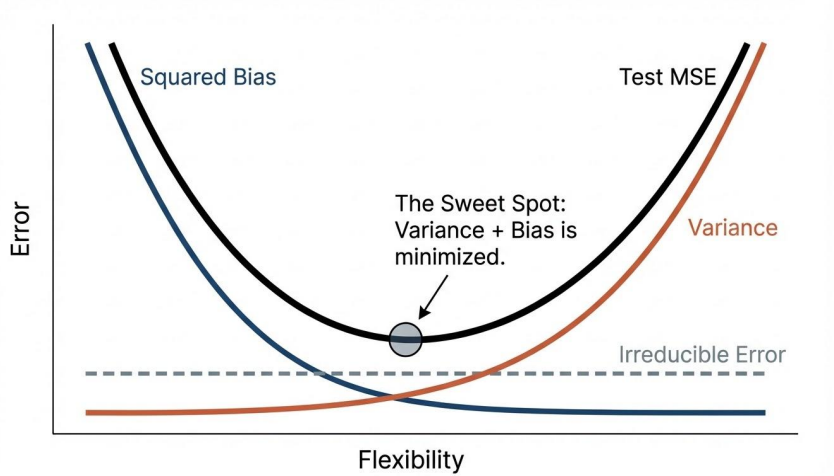
  - In particular we are interested in measuring the MSE on **test data** → data that was not used to build $\hat{f}$
    - Interested to estimate accuracy on future data, not the data we know already

- That is a measurement of the **distance between the prediction and the label**

MSE decomposition: bias-variance trade-off

$$E\left(y_0 - \hat{f}(x_0)\right)^2 = \mathrm{Var}(\hat{f}(x_0)) + [\mathrm{Bias}(\hat{f}(x_0))]^2 + \mathrm{Var}(\epsilon)$$

- **Bias**: error deriving from the over simplification of the real problem
  - Approximation of a complex problem with a simple model
- **Variance**: how much my $\hat{f}$ would change if I used a different training dataset
- ϵ: irreducible error, physical limit we cannot surpass with the available dataset



Parametric vs non-parametric methods

- Parametric methods **assume known data distribution** and a **fixed set of parameters**
  - More interpretable, but rigid
  - Effective on small, well behaved datasets
  - Faster
  - Risk: if the model is not correct may lead to high bias
  - Example: linear regression

- Non-parametric models make **no assumptions about the data distribution**
  - Flexible
  - Require larger dataset for high accuracy and higher computation
  - Generally more robust to outliers
  - Risk: may lead to overfitting (high variance)
  - Example: clustering methods

How do we measure the “badness” of our model?

The loss function

- Prince, cap.5

Loss as distance between points

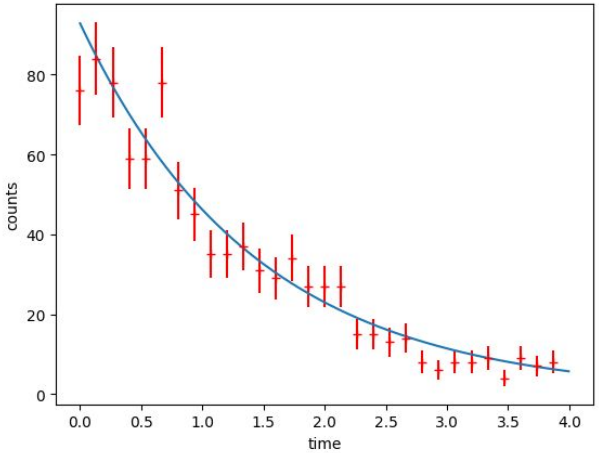
- A Loss Function L(θ) **maps the discrepancy between the predicted value ŷ and the true physics label y** into a single scalar value

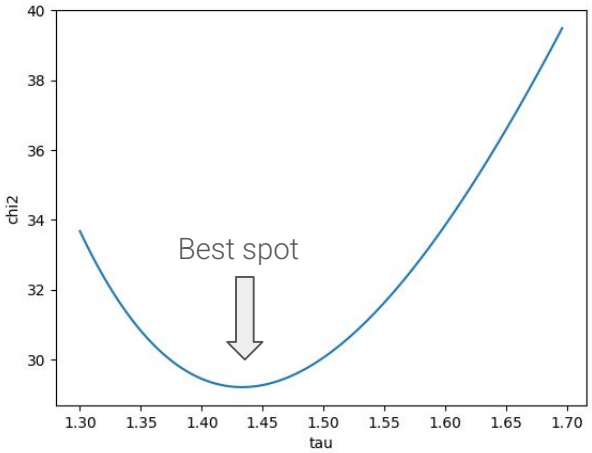
- In classical regression we find the set of params that minimize the distance between points or bins and function (least squares)
  - That's what is happening [every time you perform a fit]

Simple 1D example:
- Radioactive decay
- Described by falling exp

$$P(t) = Ae^{-t/\tau}$$

- Find tau parameter that better describes the data
- **Chi2 is our loss**





Beyond residuals:
loss as maximization of the probability of the observation

- Compute and maximize the conditional probability Pr(y|x) over possible outputs y given input x
  - **Find parameters that make the observed data most probable**

- In practice:
  - First, we **choose a parametric distribution Pr(y|θ)** defined on the output domain y
  - Set your model to **predict one or more of the parameters θ** so θ= f[x,ϕ] and Pr(y|θ) = Pr(y|f[x,ϕ])
  - **Choose the model parameters ϕ so that they maximize the combined probability** across all training examples

$$\begin{aligned}\hat{\boldsymbol{\phi}} &= \underset{\boldsymbol{\phi}}{\operatorname{argmax}}\left[\prod_{i=1}^{I} Pr(\mathbf{y}_i|\mathbf{x}_i)\right] \\ &= \underset{\boldsymbol{\phi}}{\operatorname{argmax}}\left[\prod_{i=1}^{I} Pr(\mathbf{y}_i|\boldsymbol{\theta}_i)\right] \\ &= \underset{\boldsymbol{\phi}}{\operatorname{argmax}}\left[\prod_{i=1}^{I} Pr(\mathbf{y}_i|\mathbf{f}[\mathbf{x}_i,\boldsymbol{\phi}])\right]\end{aligned}$$

That's the **maximum likelihood criterion** assuming independent inputs and identically distributed data

Regression: deriving Mean Squared Error (MSE)

- Assumption: **“noise” in our data follows a gaussian distribution**
- Then: $y_i = f(x_i, \theta) + \epsilon_i$ where $\epsilon \sim \mathcal{N}(0, \sigma^2)$
- The probability of observing y_i is $Pr(y_i|x_i, \theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - f(x_i, \theta))^2}{2\sigma^2}\right)$
- The negative log-likelihood (NLL) becomes:

$$L(\theta) = -\log \mathcal{L}(\theta) = \sum_{i=1}^{n} \left[\frac{1}{2}\log(2\pi\sigma^2) + \frac{(y_i - f(x_i, \theta))^2}{2\sigma^2}\right]$$

- Assuming constant errors, we can minimize:

$$\text{Loss}_{MSE} \propto \sum_{i=1}^{n} (y_i - f(x_i, \theta))^2$$

MSE looks for the center of the gaussian → aims at the true value of the measurement

Binary classification: deriving Binary Cross-Entropy (BCE)

- Assumption: **the stats describing the phenomenon is a Bernoulli**
- The model predicts the probability of observing $y_i \in \{0,1\}$:

$$Pr(y_i|x_i,\theta) = \hat{y}_i^{y_i}(1-\hat{y}_i)^{1-y_i}$$

- The negative log-likelihood becomes:

$$L(\theta) = -\log \prod_{i=1}^{n} \left[\hat{y}_i^{y_i}(1-\hat{y}_i)^{1-y_i}\right]$$

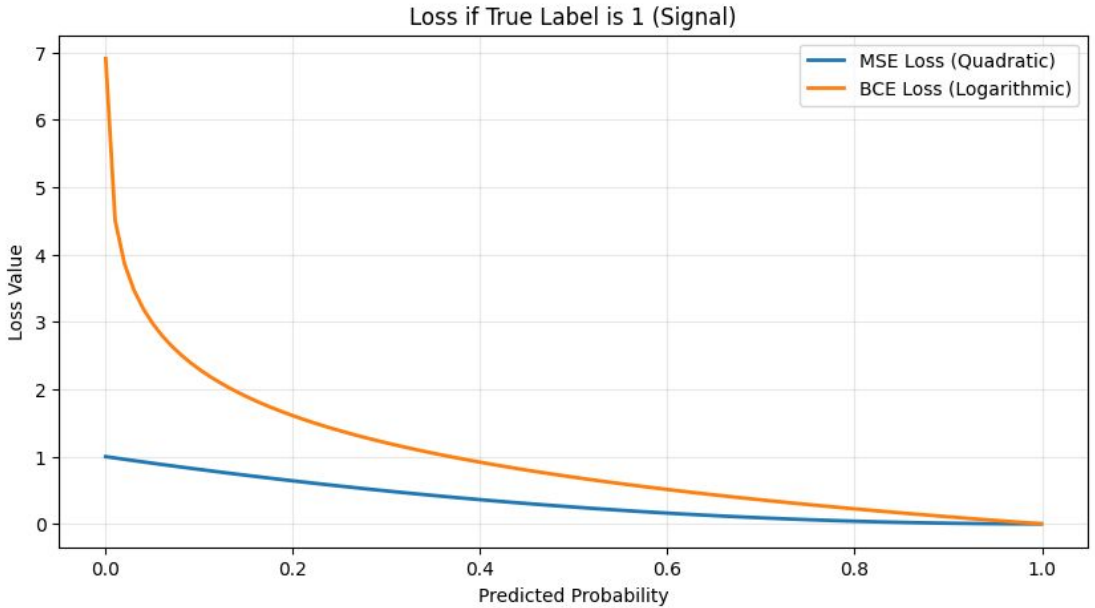
- With the logarithm properties:

$$\text{Loss}_{BCE} = -\sum_{i=1}^{n} \left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right]$$

BCE tries to separate at best the two populations by punishing the wrong predictions logarithmically

Binary classification: BCE vs MSE

- Toy model: signal vs Background separation using a single variable
- **BCE is much more aggressive than MSE in correcting errors**
  - The loss explodes if the predicted value is wrong
  - See this notebook



Hands on…

The Polynomial Overfitting Challenge

- **Complexity is a double-edged sword:** how many degrees of freedom does the Physics actually need?

- Let's take a simple **synthetic dataset** that includes the following features for **240 stars**:
  - Physical features: temperature, luminosity (L/L0), Radius (R/R0), abs magnitude
  - Categories: star color, spectral class

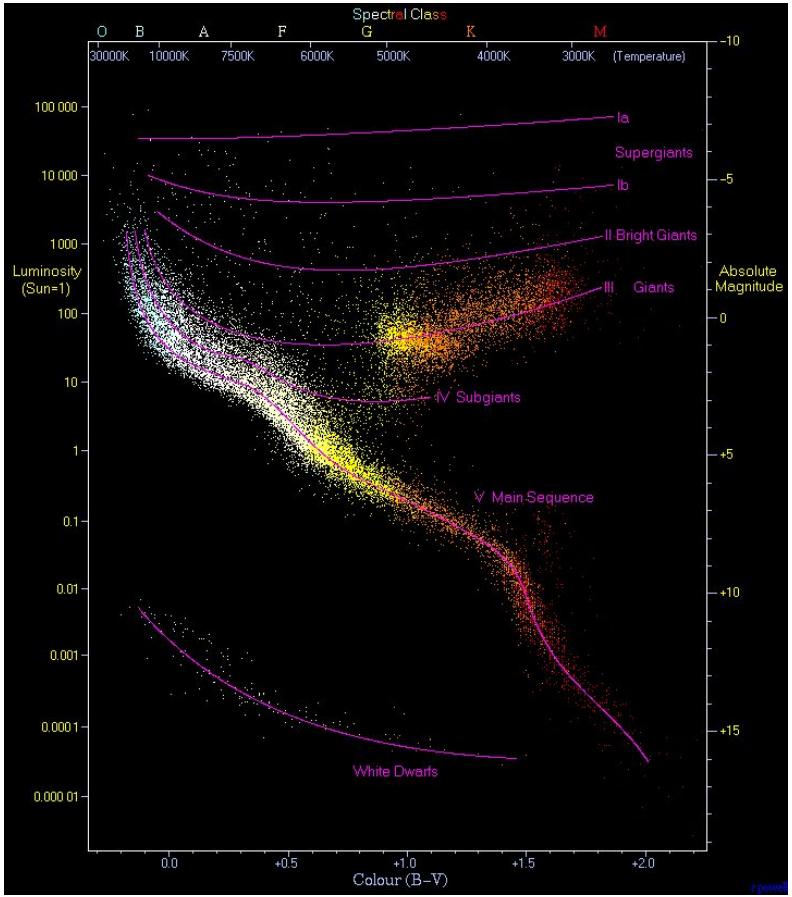
- The stefan-boltzmann law relates the thermal emission of a black body to its temperature: $\mathrm{u} = \sigma \mathrm{T}^4$
  - If we apply it to a spherical black body we can write $\mathrm{L} = 4\pi \mathrm{R}^2 \sigma \mathrm{T}^4$
    - L = luminosity
    - R = radius of the star
    - σ = Stefan-Boltzmann constant ($5.67 \times 10^{-8} \mathrm{Wm}^{-2}\mathrm{K}^{-4}$)
    - T = temperature

- Let's select the stars from the main sequence (Star type == 3)
  - What's the best model that describes **Luminosity vs Temperature**?
  - How we figure out if/when the model start **overfitting**?

The stars main sequence

- Stars belonging to the main sequence follow the Stefan-Boltzmann law since they are in thermal and hydrostatic equilibrium
  - Radius and temperature are constants
  - Radius and temperature are proportional

- Expect to find $\mathrm{L} \propto \mathrm{T}^{\mathrm{b}}$ with b between 5 and 7



Occam’s razor in statistics

- **Favors simpler models with fewer parameters** over complex ones when both fit the data equally well

- **It mitigates overfitting**, where high-variance, complex models interpret noise as underlying patterns, reducing predictive performance on new data
    - **Generalization**: improves the ability of the model to perform well on unseen data

- Physicists have a prior that says nature is usually "smooth" and "simple"
- How do we force the mathematics to prefer "smooth" curves even if we use complex models?
    - Regularization to penalize complexity → next lecture